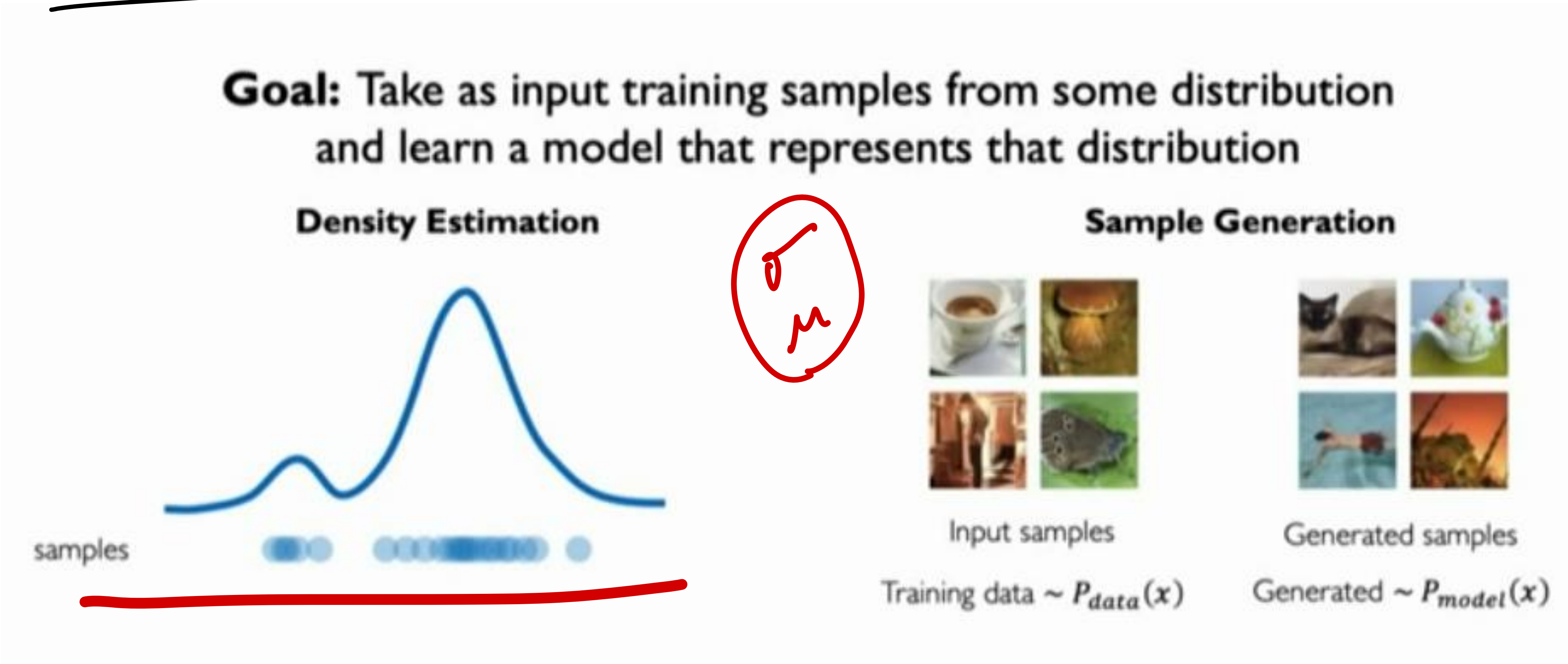
Autoencoders

Introduction

- Recall: Supervised vs. unsupervised learning

Supervised Learning	Unsupervised Learning
Data: (x, y) x is data, y is label	**Data:** x x is data, no labels!
Goal: Learn function to map $x \rightarrow y$	**Goal:** Learn the *hidden* or *underlying structure* of the data
Examples: Classification, regression, object detection, semantic segmentation, etc.	**Examples:** Clustering, feature or dimensionality reduction, etc.

Generative Modeling



σ μ

Why generative models? Debiasing

Capable of uncovering **underlying features** in a dataset

Homogeneous skin color, pose

vs

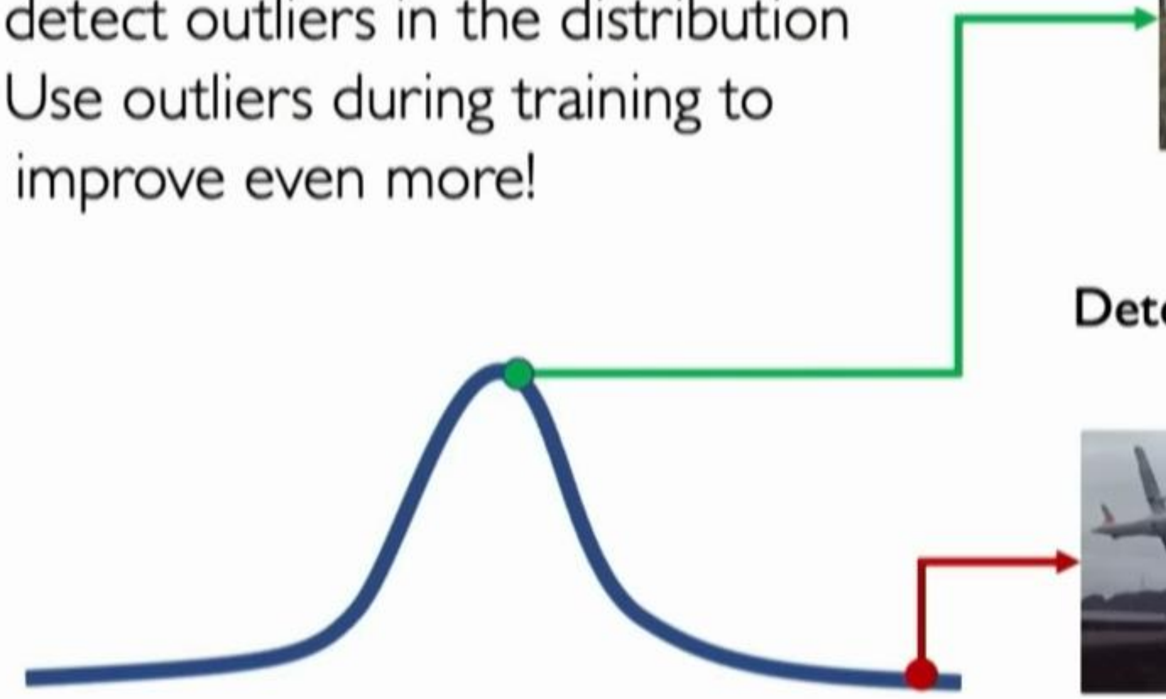
Diverse skin color, pose, illumination

How can we use this information to create fair and representative datasets?

Why generative models? Outlier detection

- **Problem:** How can we detect when we encounter something new or rare?
- **Strategy:** Leverage generative models, detect outliers in the distribution
- Use outliers during training to improve even more!

95% of Driving Data:
(1) sunny, (2) highway, (3) straight road



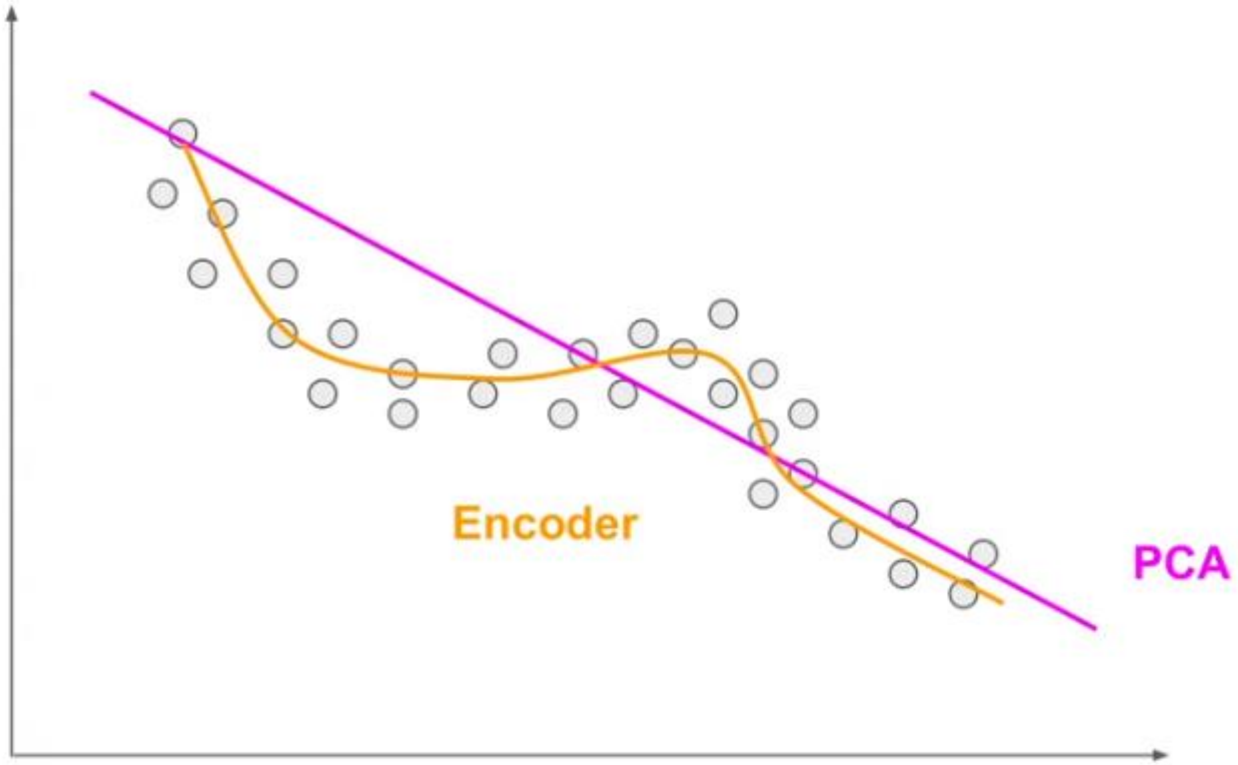
Detect outliers to avoid unpredictable behavior when training

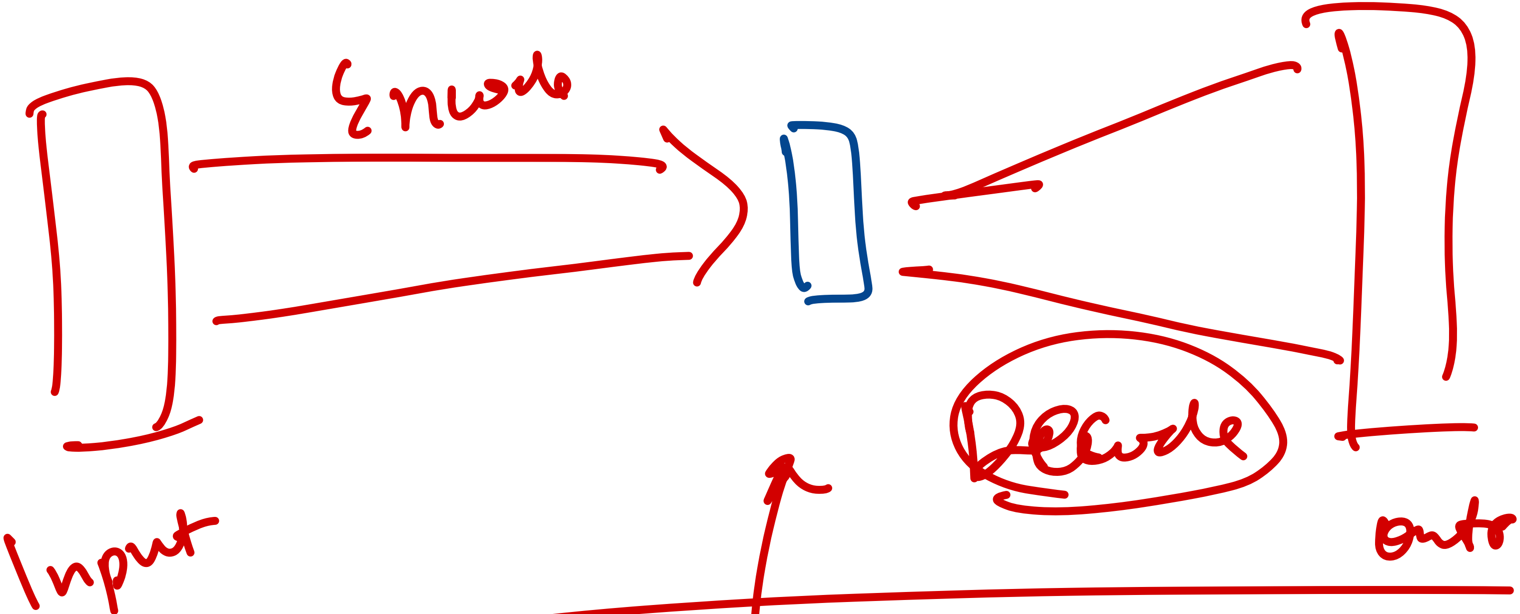
Edge Cases

Harsh Weather

Pedestrians

PCA vs Encoders





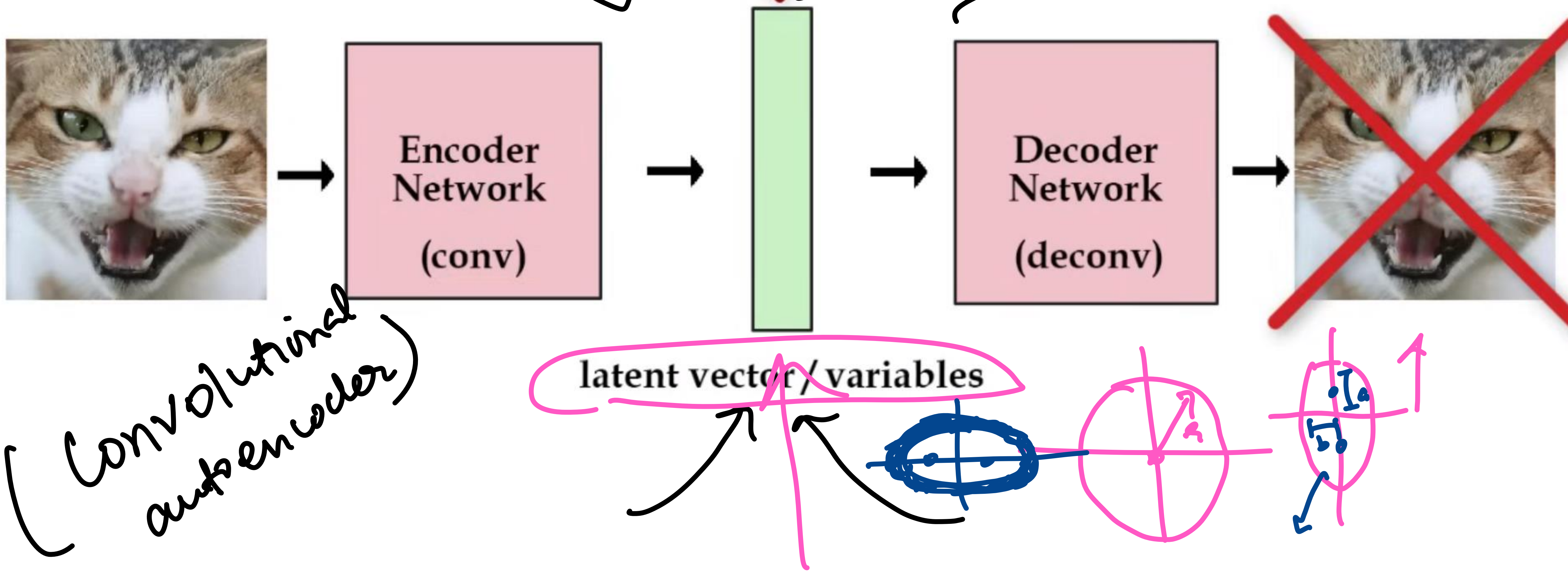
(Compress data to lower dimensions)

Latent space / variables / vector

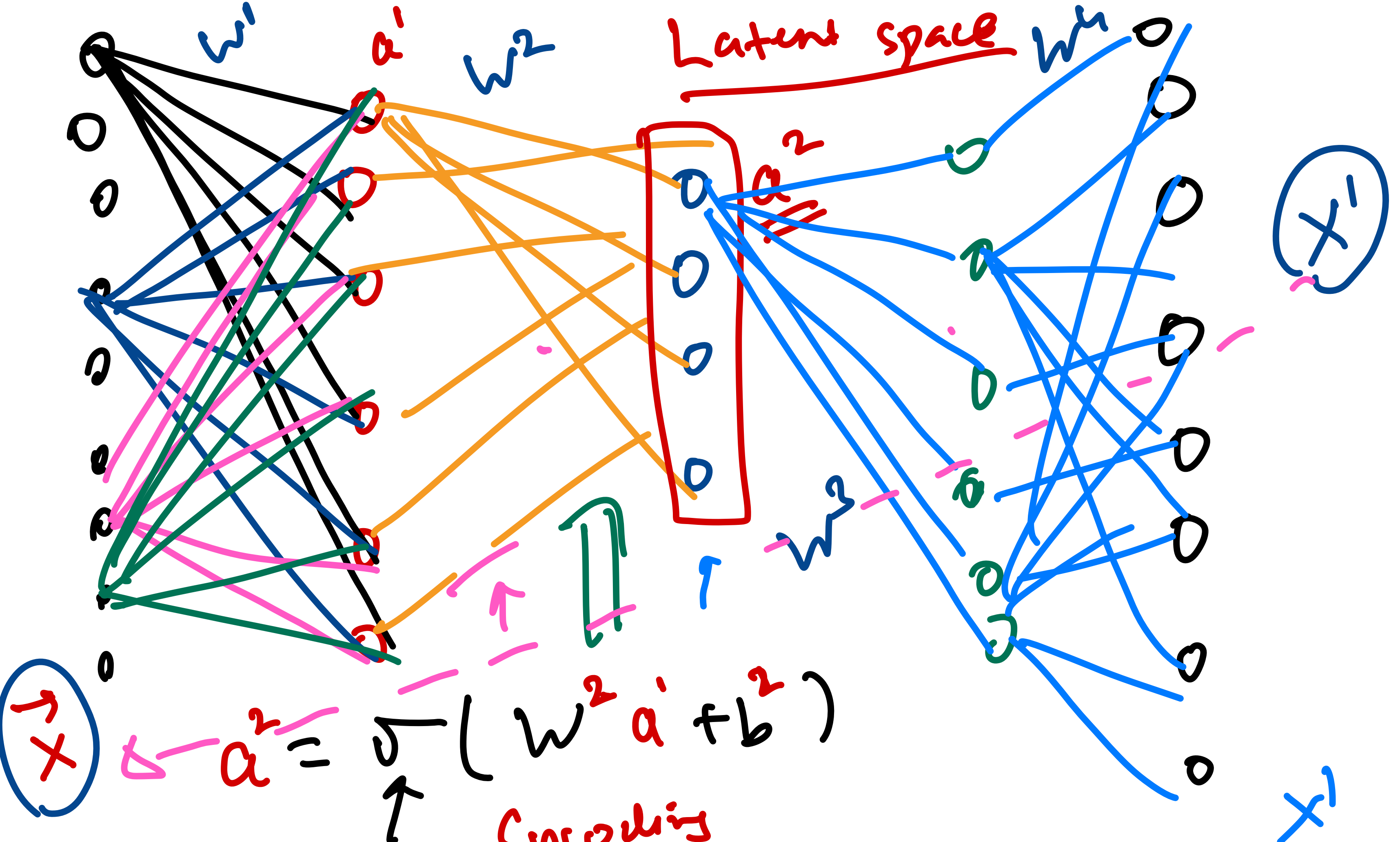
encoding

decoding

1. Why are we doing this?



(Convolutional autoencoder)



$$a^2 = \sigma(W^2 a^1 + b^2)$$

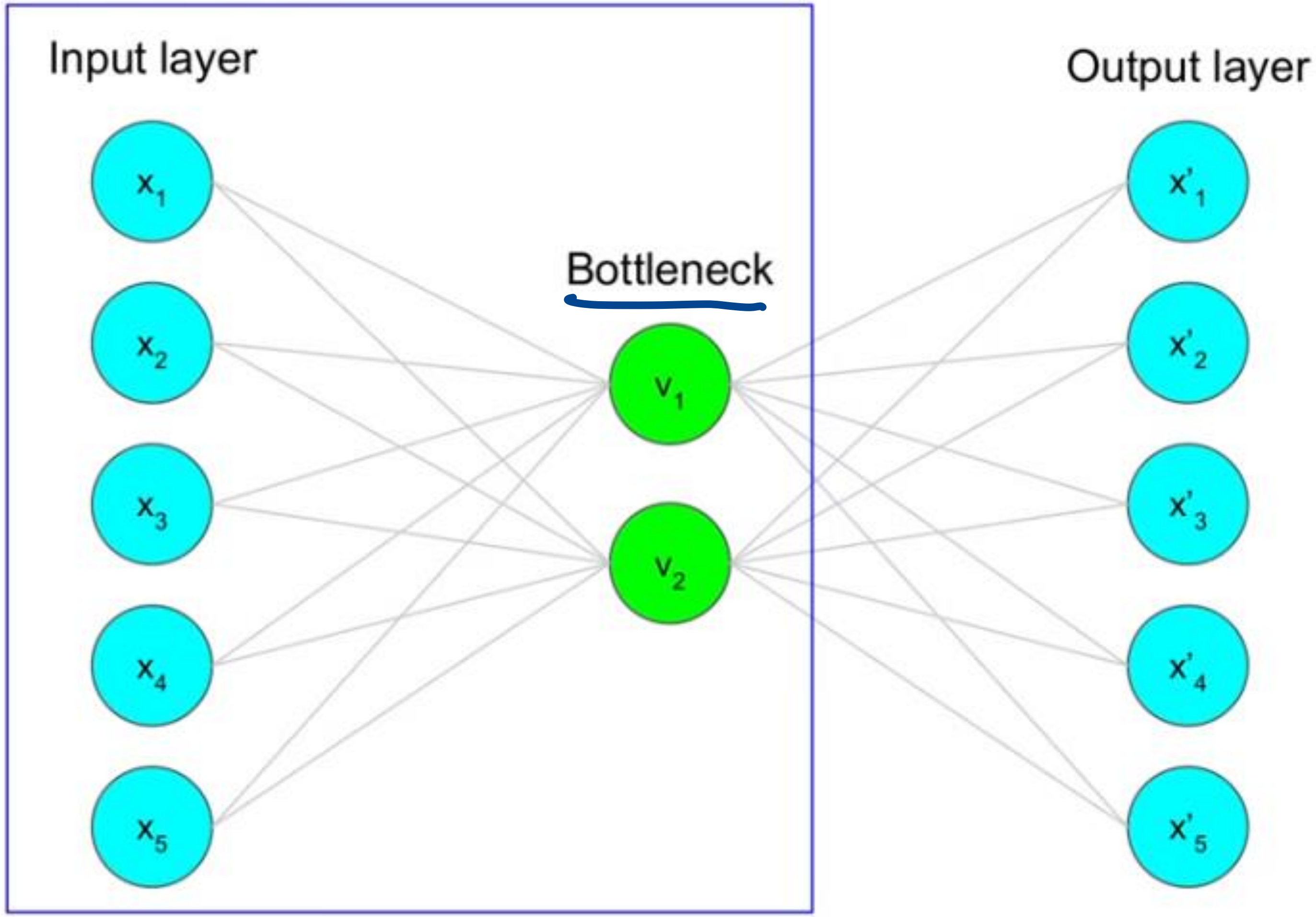
Encoding

You want

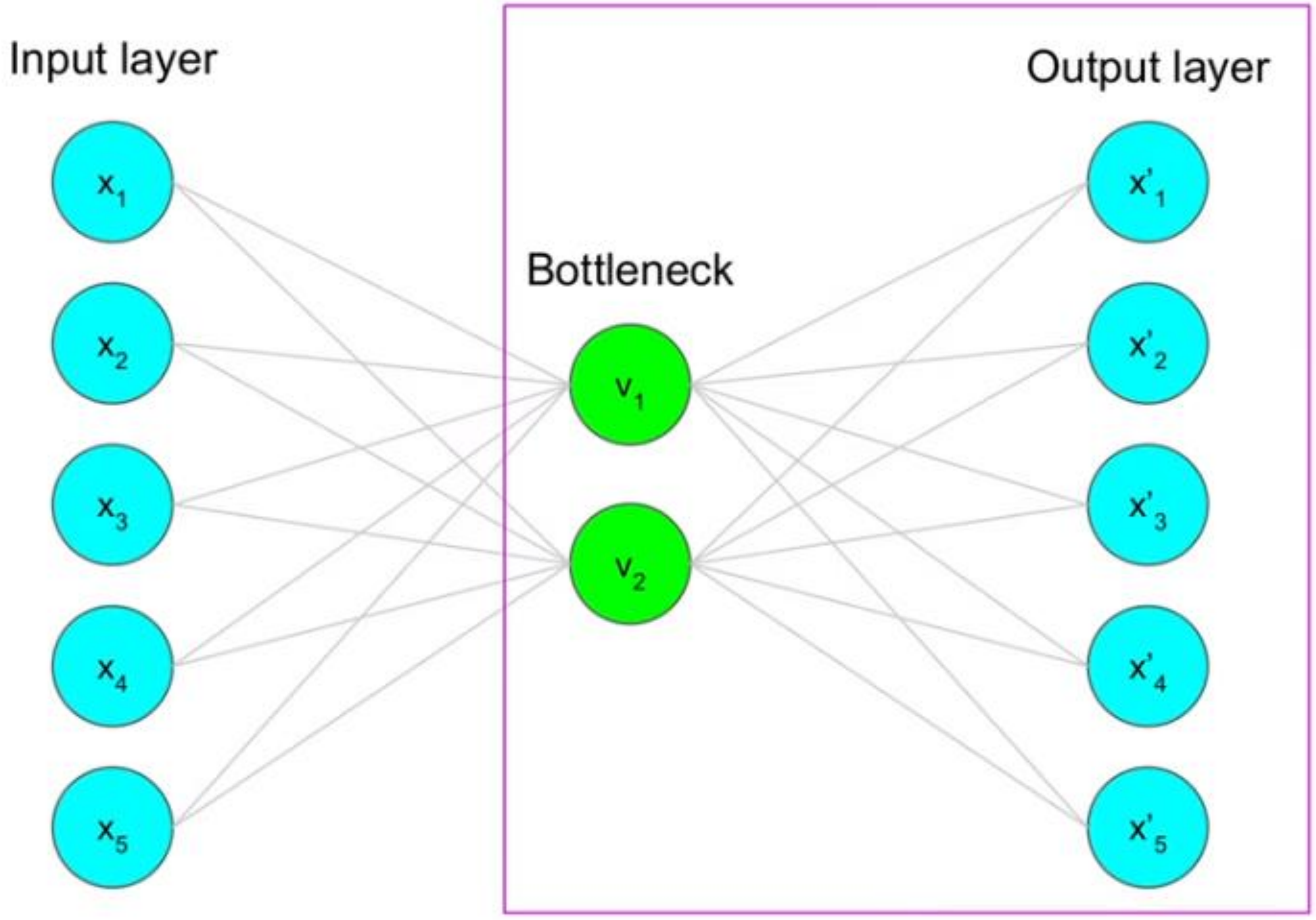
$x' \simeq x$

$L = \|x - x'\|^2$

$L \cong 0$



Encoder = compress data into lower-dimensional representation



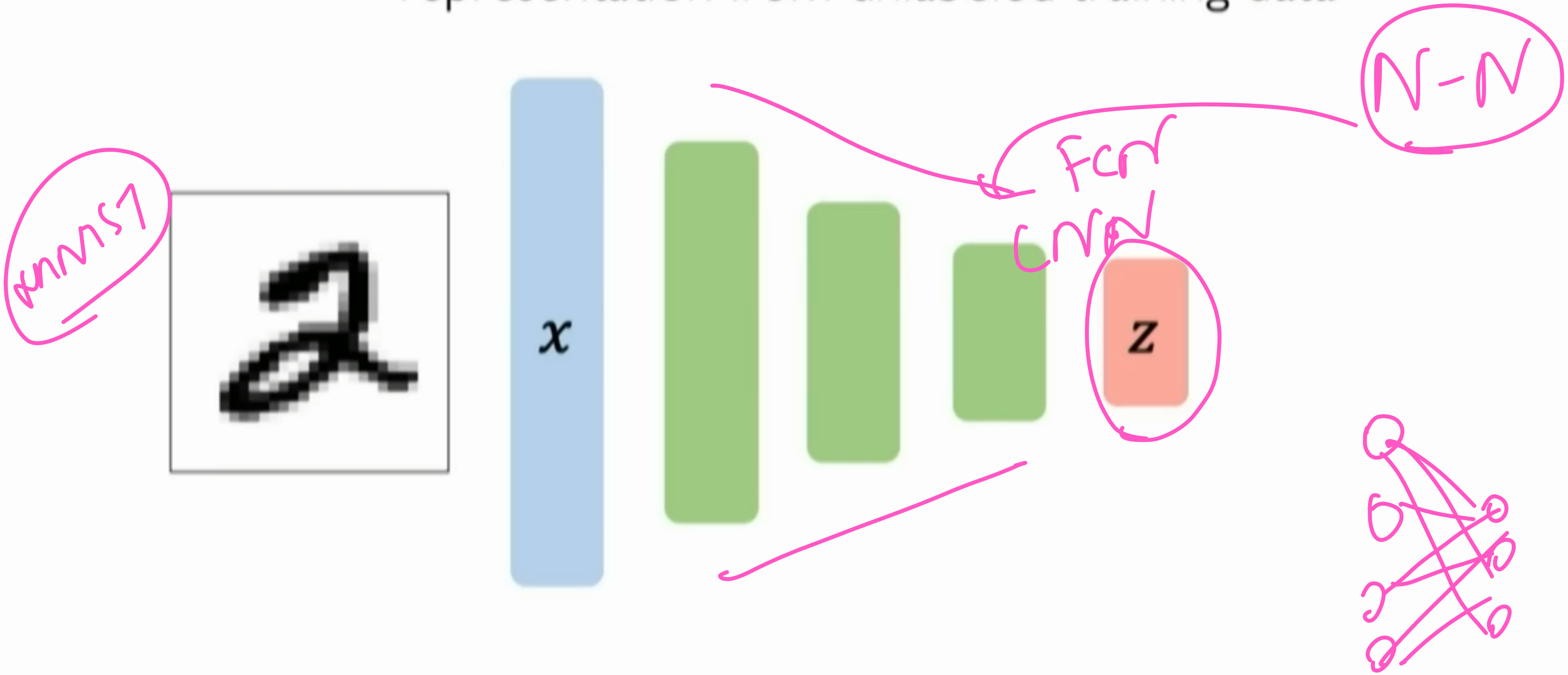
Decoder = Decompress representation back

$$E(\boxed{x}, \boxed{\hat{x}})$$

Original data Reconstructed data

Autoencoders: background

Unsupervised approach for learning a **lower-dimensional** feature representation from unlabeled training data

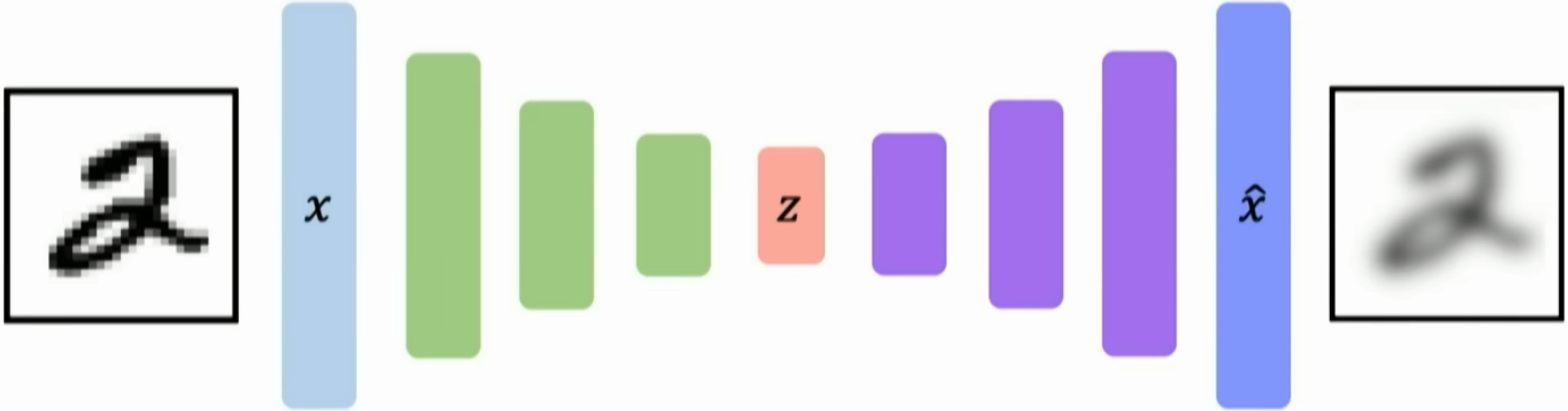


"Encoder" learns mapping from the data, x, to a low-dimensional latent space, z

Autoencoders: background

How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



"Decoder" learns mapping back from latent space, z,
to a reconstructed observation, $\hat{x}$

$\mathcal{L}(x, \hat{x}) = ||x - \hat{x}||^2$

Dimensionality of latent space → reconstruction quality

Autoencoding is a form of compression!
Smaller latent space will force a larger training bottleneck

2D latent space

5D latent space

Ground Truth

Autoencoders for representation learning

Bottleneck hidden layer forces network to learn a compressed latent representation

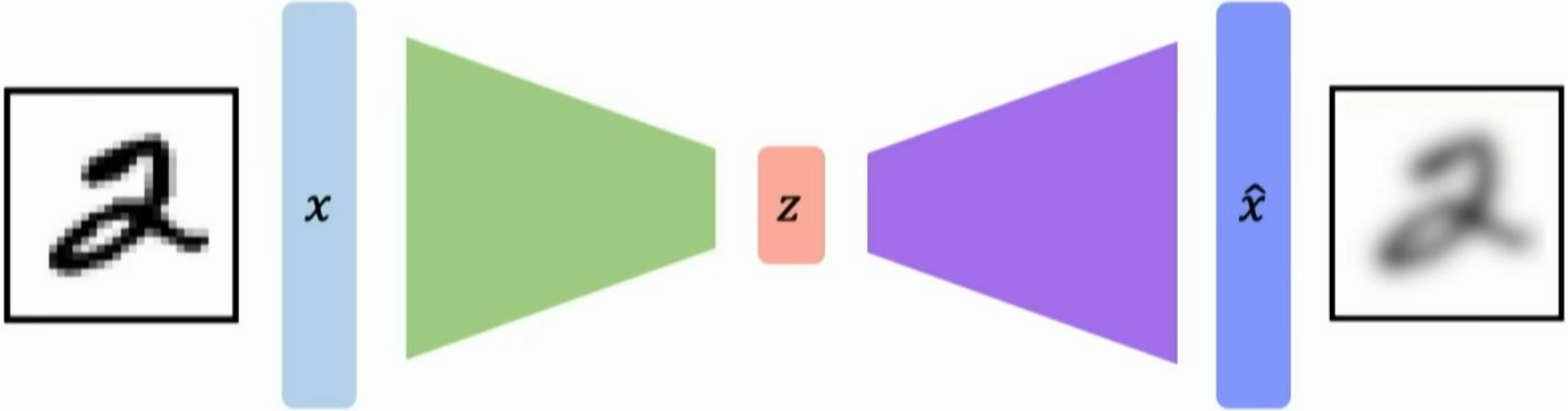
Reconstruction loss forces the latent representation to capture (or encode) as much "information" about the data as possible

Autoencoding = **Auto**matically **encoding** data; "Auto" = **self**-encoding

Deterministic

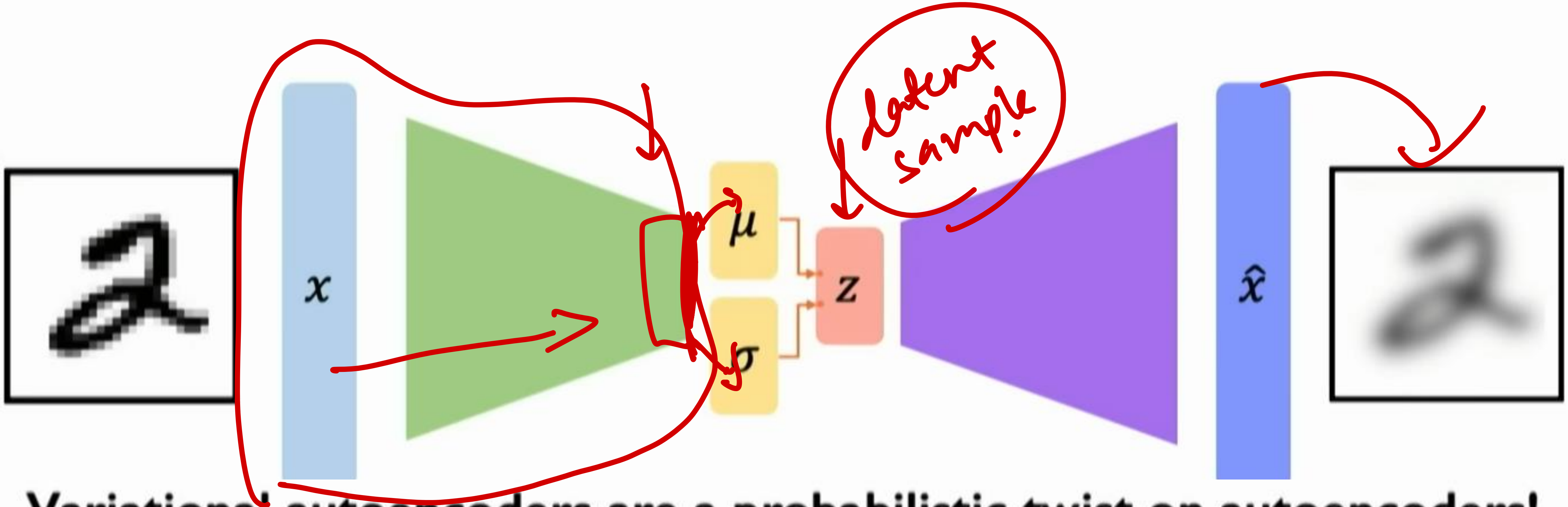
Variational

Traditional autoencoders



Probabilistic

VAEs: key difference with traditional autoencoder

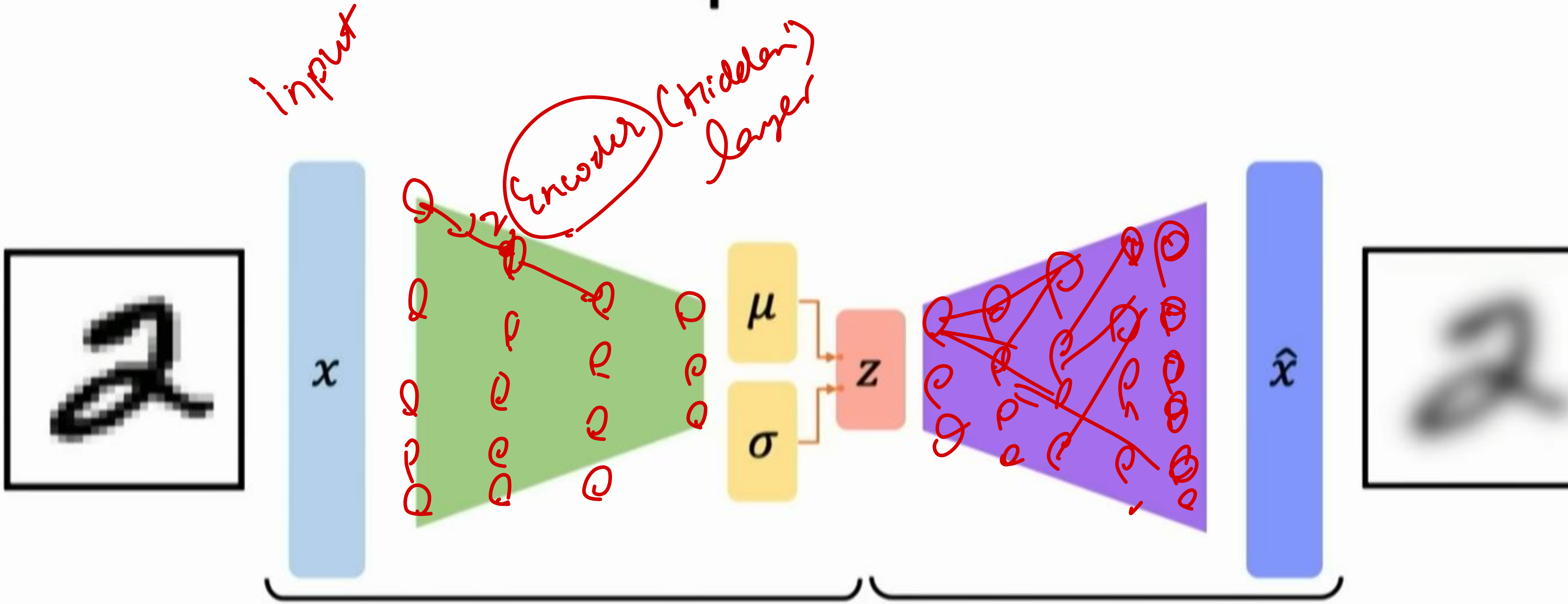


Variational autoencoders are a probabilistic twist on autoencoders!

Sample from the mean and standard deviation to compute latent sample

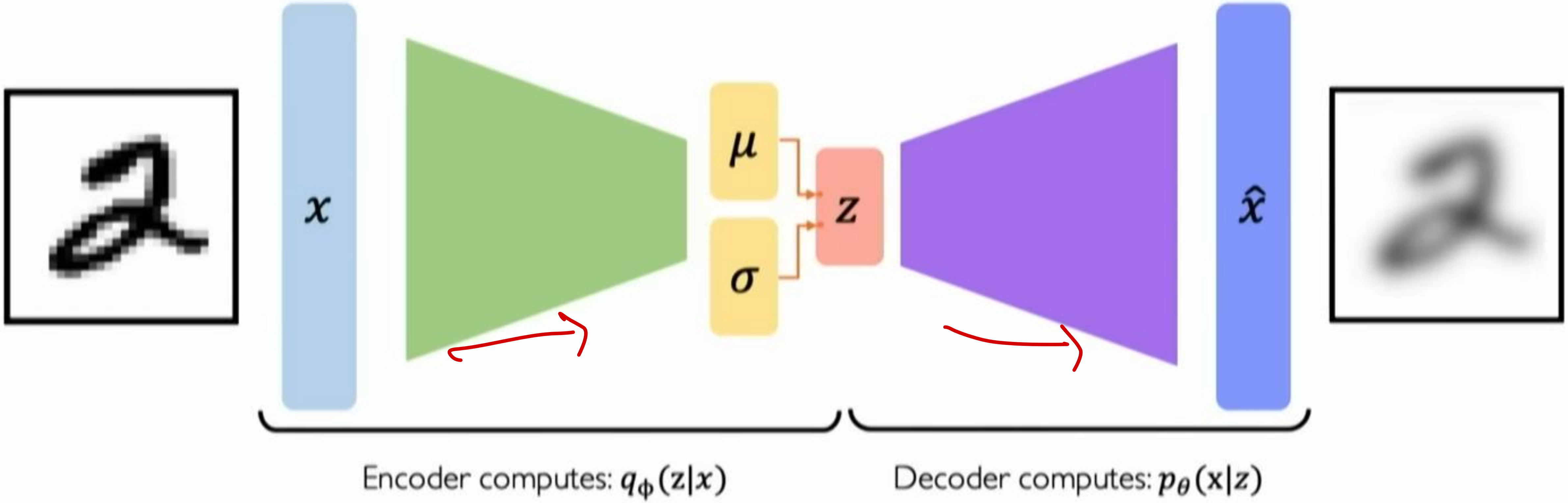
VAE optimization

NN



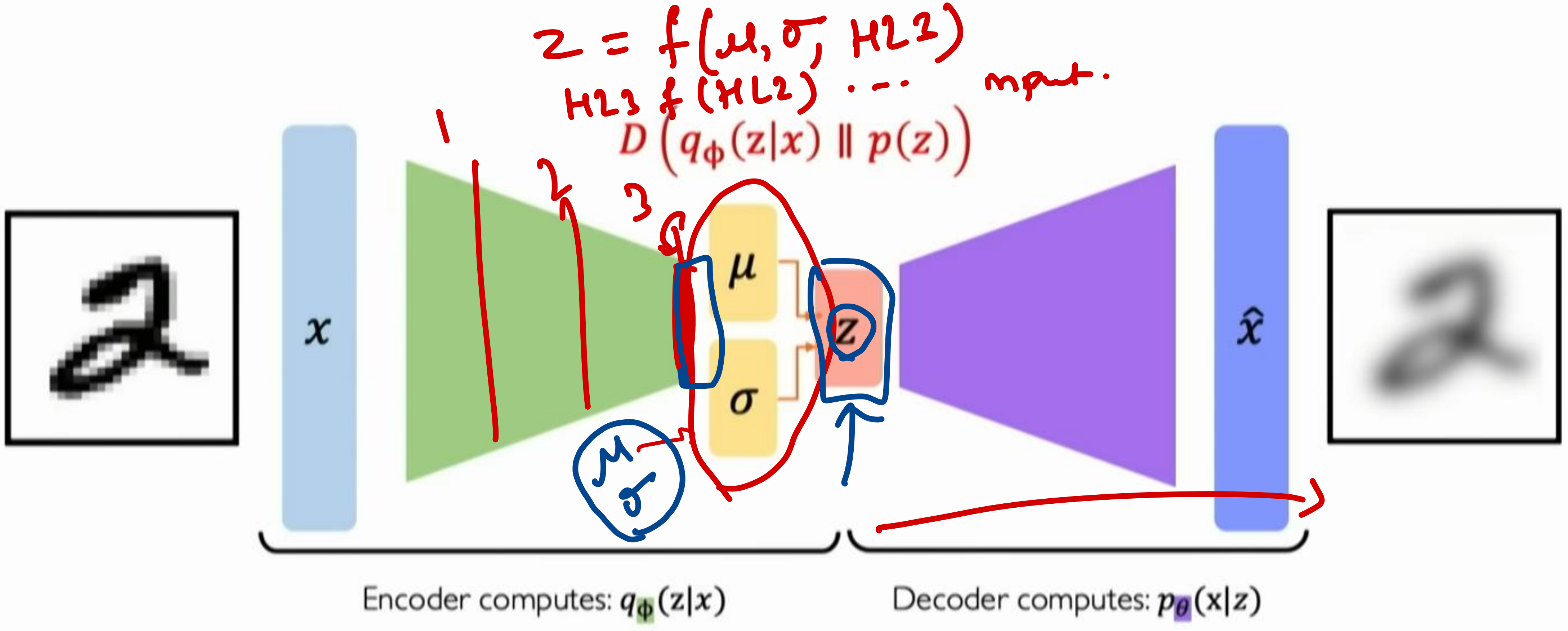
Encoder computes: $q_\phi(z|x)$ Decoder computes: $p_\theta(x|z)$

VAE optimization



$$\mathcal{L}(\phi, \theta, x) = \text{(reconstruction loss)} + \text{(regularization term)}$$

VAE optimization



$\mathcal{L}(\phi, \theta, x) = \text{(reconstruction loss)} + \boxed{\text{(regularization term)}}$

Priors on the latent distribution

$$\mathrm{KL}(q(z|x)||p(z)) = \frac{1}{2}\sum_{i=1}^{K}\left(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2\right)$$

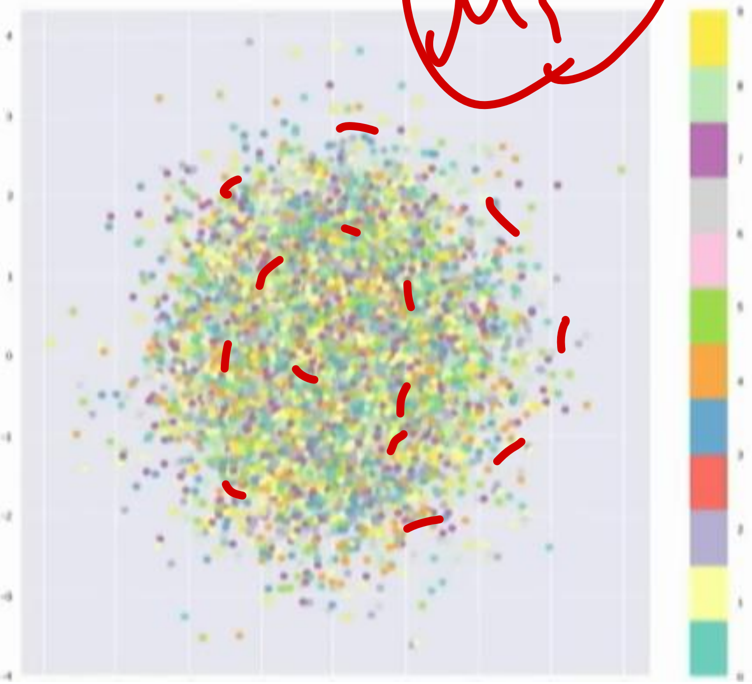
Kullback Leibler loss

$$D\left(q_\phi(z|x) \parallel p(z)\right)$$

Inferred latent distribution

Fixed prior on latent distribution

μ, σ



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- Encourages encodings to distribute encodings evenly around the center of the latent space
- Penalize the network when it tries to "cheat" by clustering points in specific regions (i.e., by memorizing the data)

latent

$$\frac{1}{2}\sum_{j=0}^{k}\left(\sigma_j + \mu_j^2 - 1 - \log(\sigma_j^2)\right)$$

Standard dev. mean

(of the output of the encoder.)

$$\frac{\mu}{\sigma}$$

dimensionality

$$\frac{1}{2}\sum_{j=0}^{k}\left(\sigma_j + \mu_j^2 - 1 - \log\sigma_j\right)$$

s.dev

mean of the

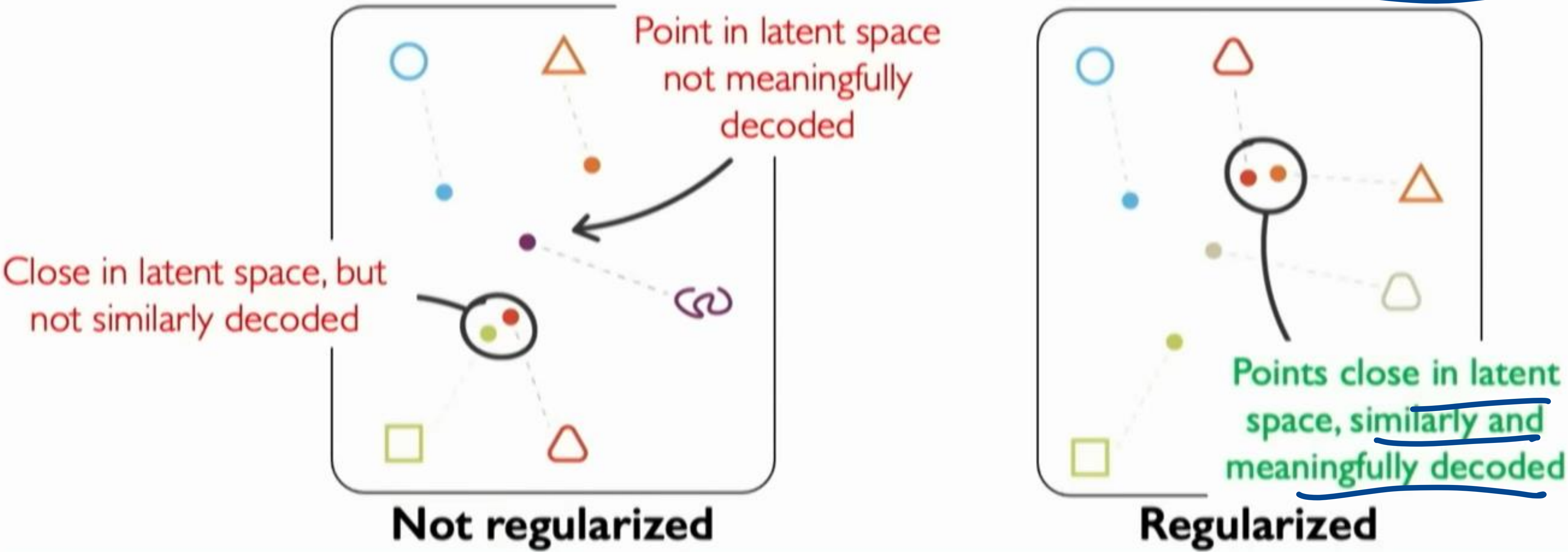
output of encoder

$\sim$

Intuition on regularization and the Normal prior

What properties do we want to achieve from regularization? 🤔

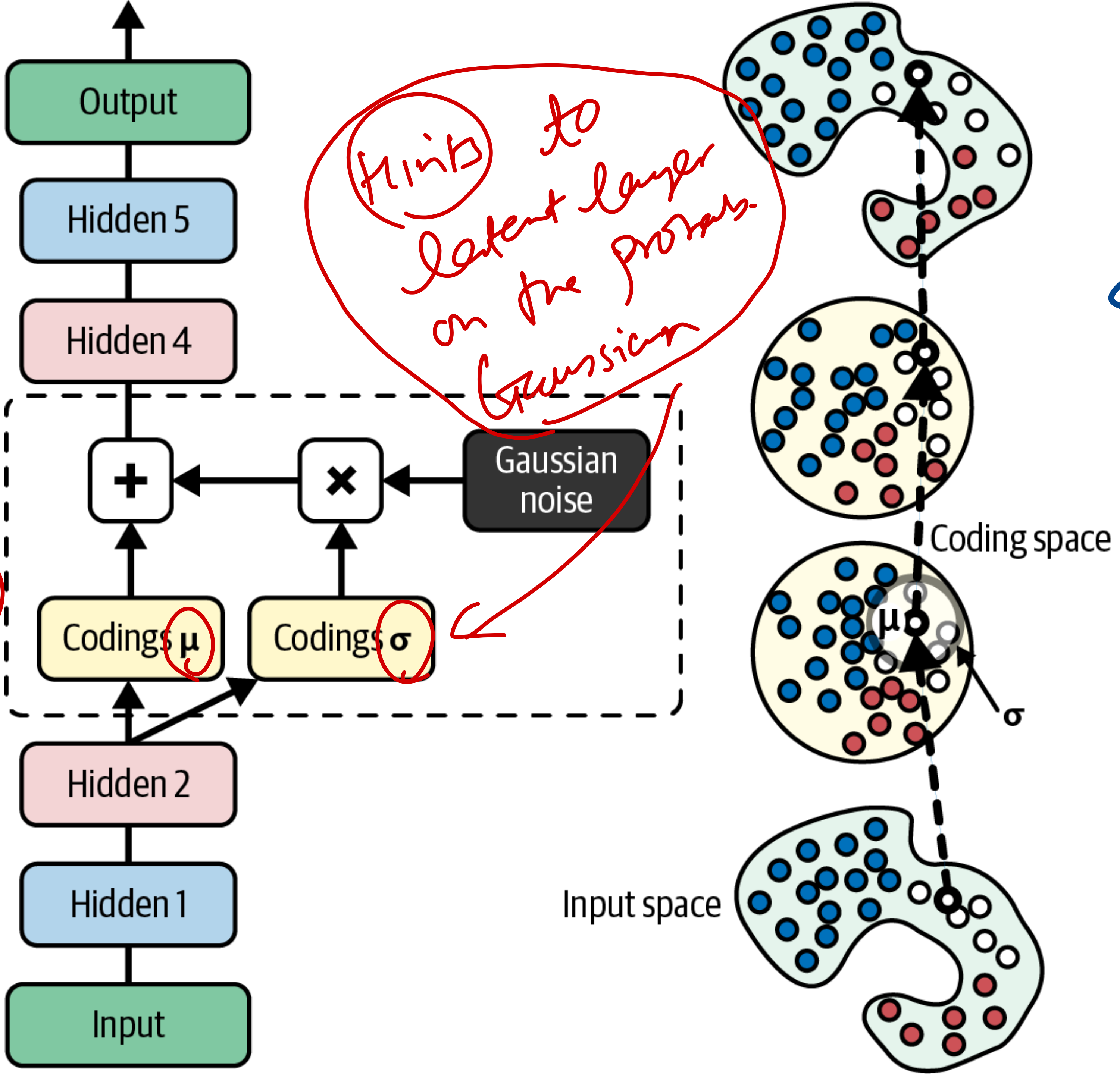
1. **Continuity:** points that are close in latent space → similar content after decoding
2. **Completeness:** sampling from latent space → "meaningful" content after decoding



Read

Aspect	Autoencoder	Variational Autoencoder (VAE)
Objective	Learn a compressed representation of the input data without any probabilistic assumptions.	Learn a probabilistic model of the input data by modeling the latent space as a probability distribution.
Latent Space	Deterministic, each input is mapped to a single point in the latent space.	Probabilistic, each input is mapped to a probability distribution in the latent space.
Encoder Output	Output is a fixed-size encoding vector.	Output consists of the mean and variance parameters of a probability distribution.
Decoder Input	Receives fixed-size encoding vector.	Receives sampled values from the latent space distribution.
Reconstruction	Deterministic, each input maps to a single reconstructed output.	Probabilistic, reconstruction is sampled from the distribution conditioned on the input.
Training Objective	Minimize reconstruction error (e.g., mean squared error).	Maximize the evidence lower bound (ELBO), consisting of reconstruction error and KL-divergence.
Representation Learning	Capable of learning meaningful representations but not inherently interpretable.	Encourages disentangled and interpretable representations by modeling the latent space as a probability distribution.
Applications	Data compression, feature learning, denoising autoencoders, anomaly detection.	Generative modeling, image generation, unsupervised learning, representation learning.

VAE



μ, σ

Hints to latent layer on the proposals Gaussian

μ / σ

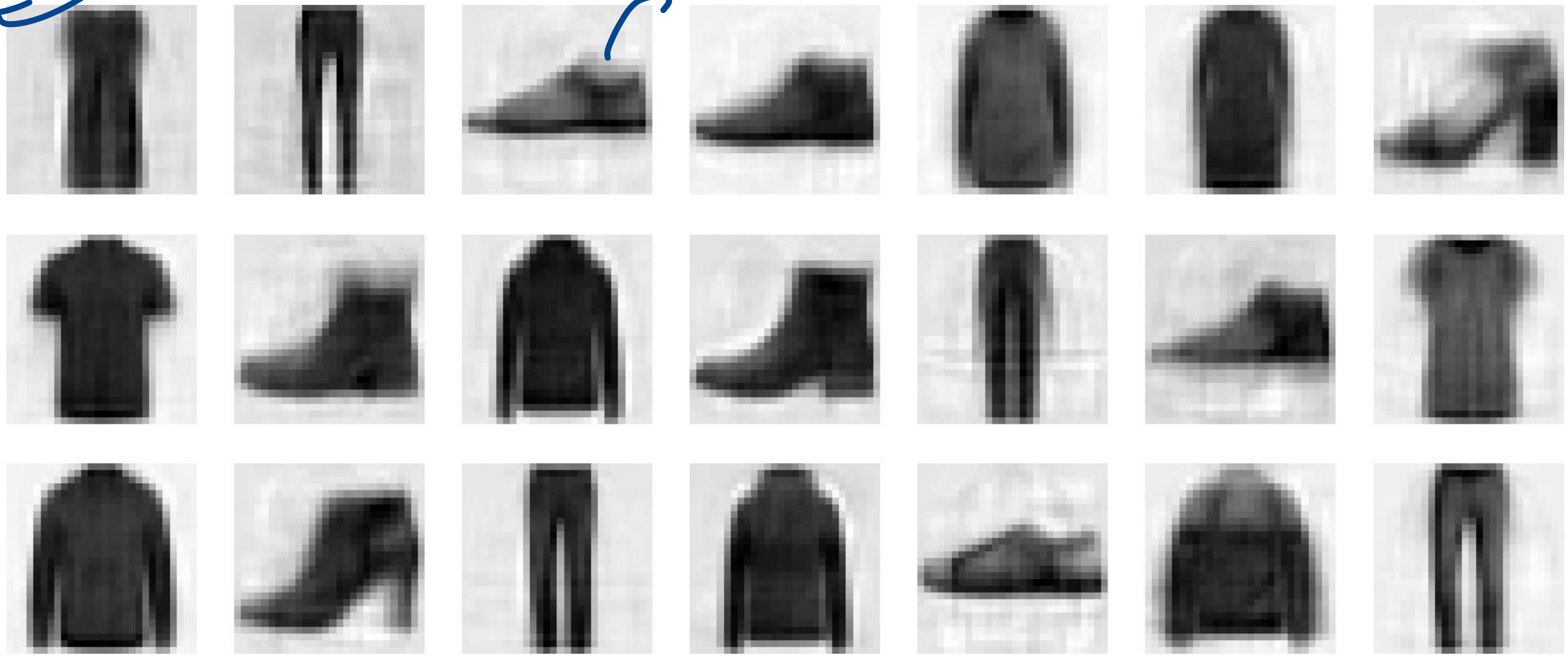
(μ, σ) VAE trained

VAE generated images from Fashion MNIST

Pg 658

μ, σ

Lat



random.normal (shape=[3*7, 10])

Other Types of Autoencoders

- Vanilla Autoencoder
- Sparse Autoencoder
- Denoising Autoencoder → Clean input from noisy input
- Variational Autoencoder
- Convolutional Autoencoder
- Recurrent Autoencoder
- Generative Adversarial Based Autoencoder

GAN

Convolutional Autoencoder

CS229

math

```
conv_encoder = tf.keras.Sequential([
    tf.keras.layers.Reshape([28, 28, 1]),
    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),  # output: 14 × 14 x 16
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),  # output: 7 × 7 x 32
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2),  # output: 3 × 3 x 64
    tf.keras.layers.Conv2D(30, 3, padding="same", activation="relu"),
    tf.keras.layers.GlobalAvgPool2D()  # output: 30
])
conv_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(3 * 3 * 16),
    tf.keras.layers.Reshape((3, 3, 16)),
    tf.keras.layers.Conv2DTranspose(32, 3, strides=2, activation="relu"),
    tf.keras.layers.Conv2DTranspose(16, 3, strides=2, padding="same",
                                    activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, 3, strides=2, padding="same"),
    tf.keras.layers.Reshape([28, 28])
])
conv_ae = tf.keras.Sequential([conv_encoder, conv_decoder])
```

Interested in More Deep Learning

- Register for the Introduction to Deep Learning Course, MIT.

- https://www.youtube.com/watch?v=xwrzh4e8DLs
- https://www.youtube.com/watch?v=fcvYpzHmhvA
- https://www.youtube.com/watch?v=3G5hWM6jqPk